

Tugas 10: Praktikum Mandiri 10

Fatih Dzakwan Susilo - 0110224235

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: fatihdzakwansusilo@gmail.com

1. Praktikum Mandiri 9

Link Github : https://github.com/FatihDzkwn/Project_Machine_Learning.git

1.1 Latihan 1

Temperatur Udara (°C)	Kecepatan Angin (km/jam)	Klasifikasi atau Persepsi Marry
10	0	Dingin
25	0	Panas
15	5	Dingin
20	3	Panas
18	7	Dingin
20	10	Dingin
22	5	Panas
24	6	Panas

Gambar 1. Data (raining).

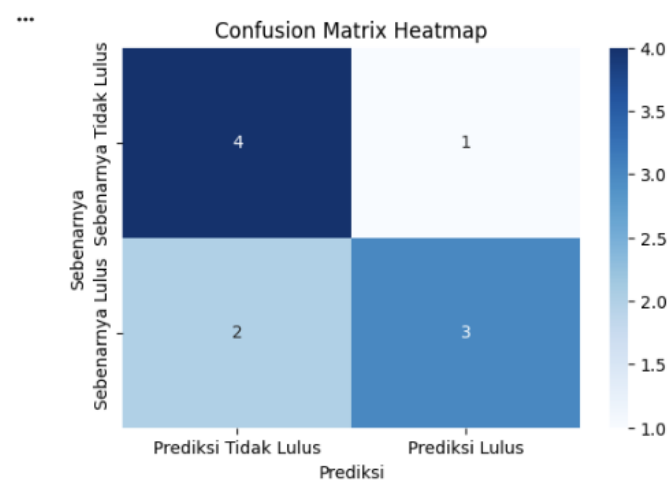
- Bagaimana persepsi Marry saat temperatur udara 16°C dan kecepatan angin 3 km/jam?
 - Dingin
- Tentukan nilai k yang tepat berdasarkan akurasi yang paling baik
 - k terbaik = 1

1.2 Latihan 2

Nim	Hasil Sebenarnya	Hasil Prediksi
TI001	Lulus	Lulus
TI002	Lulus	Lulus
TI003	Lulus	Lulus
TI004	Lulus	Tidak Lulus
TI005	Lulus	Tidak Lulus
TI006	Tidak Lulus	Lulus
TI007	Tidak Lulus	Tidak Lulus
TI008	Tidak Lulus	Tidak Lulus
TI009	Tidak Lulus	Tidak Lulus
TI010	Tidak Lulus	Tidak Lulus

Gambar 2. Data (Grade).

- Buat tabel confusion matrix berdasarkan tabel diatas



Gambar 3. Tabel Confusion Matrix.

- Hitung nilai persentase dari: accuracy, precision, dan recall
 - Accuracy = 0.70
 - Precision = 0.75
 - Recall = 0.60

1.3 Latihan 3

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Gambar 4. Mounting Google Drive.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix
from imblearn.over_sampling import SMOTE
```

Gambar 5. Import Library.

```
path = "/content/drive/MyDrive/Project_Machine_Learning/Praktikum_10/Data/"

df_test = pd.read_excel(path + "kelulusan_test.xls")
df_train = pd.read_excel(path + "kelulusan_train.xls")
```

Gambar 6. Load Dataset.

[4] ✓ 0 d

df_train.head()

	NAMA	JENIS KELAMIN	STATUS MAHASISWA	UMUR	STATUS NIKAH	IPS 1	IPS 2	IPS 3	IPS 4	IPS 5	IPS 6	IPS 7	IPS 8	IPK	STATUS KELULUSAN
0	ANIK WIDAYANTI	PEREMPUAN	BEKERJA	28	BELUM MENIKAH	2.76	2.80	3.20	3.17	2.98	3.00	3.03	0.0	3.07	TERLAMBAT
1	DWI HESTYNA PRIHASTANTY	PEREMPUAN	MAHASISWA	32	BELUM MENIKAH	3.00	3.30	3.14	3.14	2.84	3.13	3.25	0.0	3.17	TERLAMBAT
2	MURYA ARIEF BASUKI	PEREMPUAN	BEKERJA	29	BELUM MENIKAH	3.50	3.30	3.70	3.29	3.53	3.72	3.73	0.0	3.54	TERLAMBAT
3	NANIK SUSANTI	PEREMPUAN	MAHASISWA	27	BELUM MENIKAH	3.17	3.41	3.61	3.36	3.48	3.63	3.46	0.0	3.41	TERLAMBAT
4	RIFKA ISTIQFARINA	PEREMPUAN	BEKERJA	29	BELUM MENIKAH	2.90	2.89	3.30	2.85	2.98	3.00	3.08	0.0	3.09	TERLAMBAT

Langkah berikutnya: [Buat kode dengan df_train](#) [New interactive sheet](#)

Gambar 7. Menampilkan 5 baris pertama dari data training.

[5] ✓ 0 d

df_test.head()

	NAMA	JENIS KELAMIN	STATUS MAHASISWA	UMUR	STATUS NIKAH	IPS 1	IPS 2	IPS 3	IPS 4	IPS 5	IPS 6	IPS 7	IPS 8	IPK	STATUS KELULUSAN
0	UNAMA	LAKI - LAKI	MAHASISWA	24	BELUM MENIKAH	3.17	2.70	3.23	2.41	3.00	2.47	1.75	0.00	2.75	TEPAT
1	LEYLA TRIYANA PRATIWI	PEREMPUAN	MAHASISWA	26	BELUM MENIKAH	3.60	3.50	3.42	2.85	3.31	2.95	2.18	NaN	3.39	TEPAT
2	VERIS SOFIYAN PRAYOGA	LAKI - LAKI	MAHASISWA	29	BELUM MENIKAH	2.67	2.66	2.93	3.14	2.92	2.64	2.88	0.50	2.81	TEPAT
3	ADITYA AKBAR NUGRAHA	LAKI - LAKI	MAHASISWA	27	BELUM MENIKAH	2.48	2.86	2.09	2.55	2.55	2.43	2.55	2.17	2.82	TEPAT
4	ERNA EKA RIYANTI	PEREMPUAN	MAHASISWA	25	BELUM MENIKAH	3.19	3.08	3.31	2.83	3.36	2.73	3.06	0.00	3.09	TEPAT

Langkah berikutnya: [Buat kode dengan df_test](#) [New interactive sheet](#)

Gambar 8. Menampilkan 5 baris pertama dari data testing.

[6]
✓ 0 d

df_train.info()

```
*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 379 entries, 0 to 378
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   NAMA                  379 non-null    object
1   JENIS KELAMIN         379 non-null    object
2   STATUS MAHASISWA      379 non-null    object
3   UMUR                  379 non-null    int64
4   STATUS NIKAH          379 non-null    object
5   IPS 1                 379 non-null    float64
6   IPS 2                 379 non-null    float64
7   IPS 3                 379 non-null    float64
8   IPS 4                 379 non-null    float64
9   IPS 5                 379 non-null    float64
10  IPS 6                 379 non-null    float64
11  IPS 7                 379 non-null    float64
12  IPS 8                 372 non-null    float64
13  IPK                   376 non-null    float64
14  STATUS KELULUSAN      379 non-null    object
dtypes: float64(9), int64(1), object(5)
memory usage: 44.5+ KB
```

Gambar 9. Memberikan informasi tentang struktur dataset training.

[7]
✓ 0 d

df_test.info()

```
*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 145 entries, 0 to 144
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   NAMA                  145 non-null    object
1   JENIS KELAMIN         145 non-null    object
2   STATUS MAHASISWA      145 non-null    object
3   UMUR                  145 non-null    int64
4   STATUS NIKAH          145 non-null    object
5   IPS 1                 145 non-null    float64
6   IPS 2                 145 non-null    float64
7   IPS 3                 145 non-null    float64
8   IPS 4                 145 non-null    float64
9   IPS 5                 145 non-null    float64
10  IPS 6                 145 non-null    float64
11  IPS 7                 145 non-null    float64
12  IPS 8                 141 non-null    float64
13  IPK                   142 non-null    float64
14  STATUS KELULUSAN      145 non-null    object
dtypes: float64(9), int64(1), object(5)
memory usage: 17.1+ KB
```

Gambar 10. Memberikan informasi tentang struktur dataset testing.

```
[8]
✓ 0 d # untuk melihat nilai unique
print(df_train['JENIS KELAMIN'].unique())
print(df_train['STATUS MAHASISWA'].unique())
print(df_train['STATUS NIKAH'].unique())
print(df_train['STATUS KELULUSAN'].unique())

▼ ... ['PEREMPUAN' 'LAKI - LAKI']
      ['BEKERJA' 'MAHASISWA']
      ['BELUM MENIKAH' 'MENIKAH']
      ['TERLAMBAT' 'TEPAT']
```

Gambar 11. Melihat Nilai Unik.

```
[9]
✓ 0 d # Menghapus Missing Value
df_train = df_train.dropna(subset=['IPS 8'])
df_train = df_train.dropna(subset=['IPK '])

df_test = df_test.dropna(subset=['IPS 8'])
df_test = df_test.dropna(subset=['IPK '])
```

Gambar 12. Menghapus Missing Value.

```
[10]
✓ 0 d # Menghapus Kolom Status Nikah di dataframe asli
df_test.drop(columns=['STATUS NIKAH'], inplace=True)
df_train.drop(columns=['STATUS NIKAH'], inplace=True)
```

Gambar 13. Menghapus Kolom Status Nikah.

```
[11]
✓ 0 d ##Menghapus kolom Nama, tapi bukan langsung di dataframe, kita hapus karena tidak digunakan pada analisis
df_train = df_train.drop(columns=['NAMA'])
df_test = df_test.drop(columns=['NAMA'])
df_train.head()
```

	JENIS KELAMIN	STATUS MAHASISWA	UMUR	IPS 1	IPS 2	IPS 3	IPS 4	IPS 5	IPS 6	IPS 7	IPS 8	IPK	STATUS KELULUSAN
0	PEREMPUAN	BEKERJA	28	2.76	2.80	3.20	3.17	2.98	3.00	3.03	0.0	3.07	TERLAMBAT
1	PEREMPUAN	MAHASISWA	32	3.00	3.30	3.14	3.14	2.84	3.13	3.25	0.0	3.17	TERLAMBAT
2	PEREMPUAN	BEKERJA	29	3.50	3.30	3.70	3.29	3.53	3.72	3.73	0.0	3.54	TERLAMBAT
3	PEREMPUAN	MAHASISWA	27	3.17	3.41	3.61	3.36	3.48	3.63	3.46	0.0	3.41	TERLAMBAT
4	PEREMPUAN	BEKERJA	29	2.90	2.89	3.30	2.85	2.98	3.00	3.08	0.0	3.09	TERLAMBAT

Gambar 14. Menghapus Kolom Nama.

```
[12]
✓ 0 d # Encoding data
replacements = {
    'JENIS KELAMIN': {'LAKI - LAKI': 1, 'PEREMPUAN': 0},
    'STATUS MAHASISWA': {'MAHASISWA': 0, 'BEKERJA': 1},
    'STATUS KELULUSAN': {'TERLAMBAT': 1, 'TEPAT': 0}
}
```

Gambar 15. Membuat Dictionary Replacements.

```
df_train = df_train.replace(replacements, inplace=False)
df_train.head()
```

FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version. To retain the old behavior, explicitly call 'result.infer_objects(copy=False)'. To opt-in to the future behavior

```
df_train = df_train.replace(replacements, inplace=False)
```

	JENIS KELAMIN	STATUS MAHASISWA	UMUR	IPS 1	IPS 2	IPS 3	IPS 4	IPS 5	IPS 6	IPS 7	IPS 8	IPK	STATUS KELULUSAN
0	0	1	28	2.76	2.80	3.20	3.17	2.98	3.00	3.03	0.0	3.07	1
1	0	0	32	3.00	3.30	3.14	3.14	2.84	3.13	3.25	0.0	3.17	1
2	0	1	29	3.50	3.30	3.70	3.29	3.53	3.72	3.73	0.0	3.54	1
3	0	0	27	3.17	3.41	3.61	3.36	3.48	3.63	3.46	0.0	3.41	1
4	0	1	29	2.90	2.89	3.30	2.85	2.98	3.00	3.08	0.0	3.09	1

Gambar 16. Menerapkan Encoding pada Data Training.

```
[13] df_test = df_test.replace(replacements, inplace=False)
[14] df_test.head()
```

/tmp/ipython-input-1812349956.py:13: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version. To retain the old behavior, explicitly call 'result.infer_objects(copy=False)'. To opt-in to the future behavior, call 'result.infer_objects(copy=False)'. To opt-out to the future behavior, call 'result.infer_objects(copy=False)'. To opt-in to the future behavior, call 'result.infer_objects(copy=False)'.

```
df_test = df_test.replace(replacements, inplace=False)
```

	JENIS KELAMIN	STATUS MAHASISWA	UMUR	IPS 1	IPS 2	IPS 3	IPS 4	IPS 5	IPS 6	IPS 7	IPS 8	IPK	STATUS KELULUSAN
0	1	0	24	3.17	2.70	3.23	2.41	3.00	2.47	1.75	0.00	2.75	0
2	1	0	29	2.67	2.66	2.93	3.14	2.92	2.64	2.88	0.50	2.81	0
3	1	0	27	2.40	2.86	2.09	2.55	2.55	2.43	2.55	2.17	2.82	0
4	0	0	25	3.19	3.08	3.31	2.83	3.36	2.73	3.06	0.00	3.09	0
5	1	0	24	3.10	2.98	3.17	3.25	3.41	3.08	3.43	3.00	3.23	0

Gambar 17. Menerapkan Encoding pada Data Testing.

```
[14] df_test.isnull().sum()
```

JENIS KELAMIN	0
STATUS MAHASISWA	0
UMUR	0
IPS 1	0
IPS 2	0
IPS 3	0
IPS 4	0
IPS 5	0
IPS 6	0
IPS 7	0
IPS 8	0
IPK	0
STATUS KELULUSAN	0

dtype: int64

Gambar 18. Mengecek jumlah missing values di setiap kolom pada dataset df_test.

```
[15]
✓ 0 d df_train.isnull().sum()

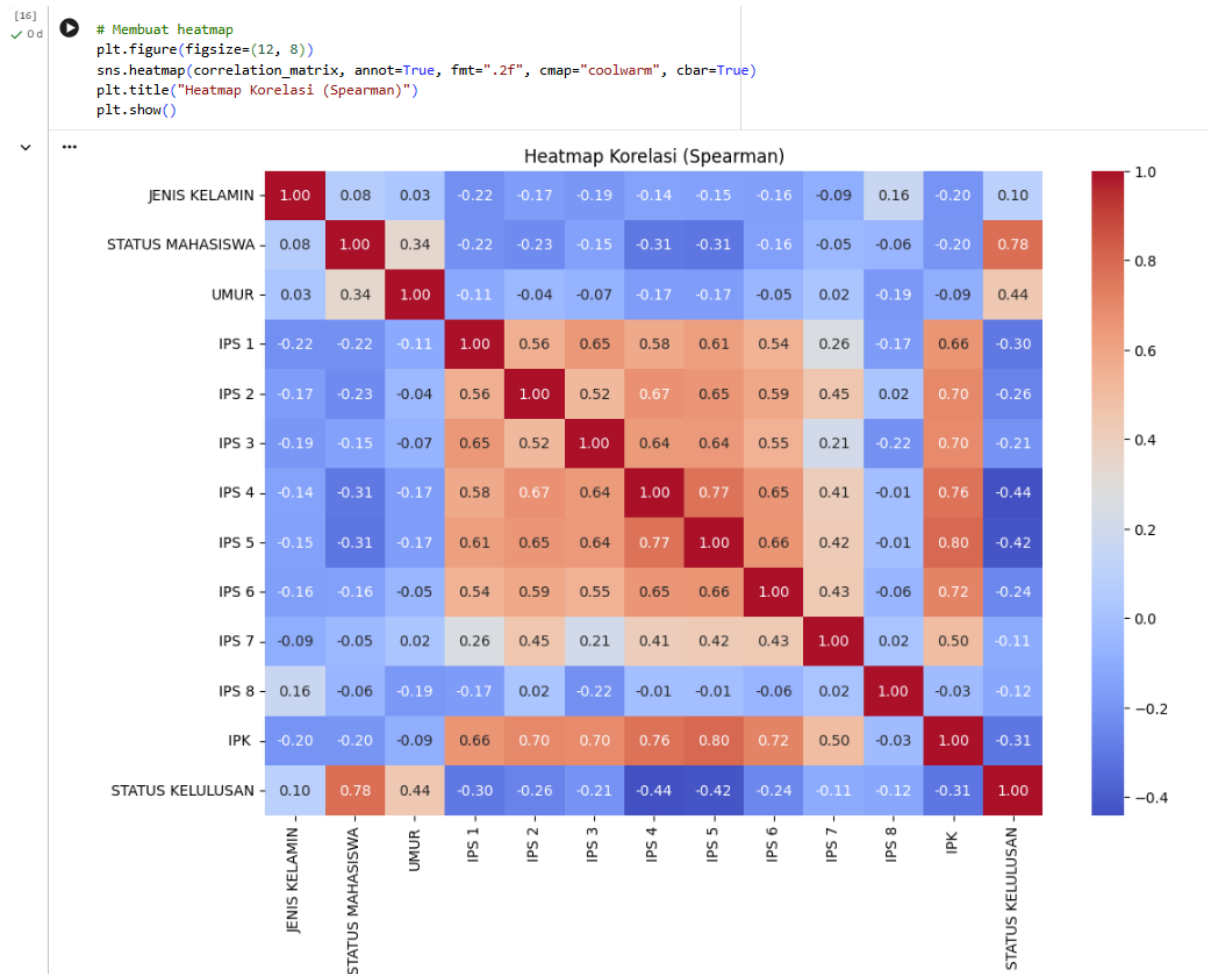
...
0
JENIS KELAMIN 0
STATUS MAHASISWA 0
UMUR 0
IPS 1 0
IPS 2 0
IPS 3 0
IPS 4 0
IPS 5 0
IPS 6 0
IPS 7 0
IPS 8 0
IPK 0
STATUS KELULUSAN 0

dtype: int64
```

Gambar 19. Mengecek jumlah missing values di setiap kolom pada dataset df_train.

```
[16]
✓ 0 d # Menghitung matriks korelasi menggunakan Spearman
correlation_matrix = df_train.corr(method='spearman')
```

Gambar 20. Menghitung Matriks Korelasi.



Gambar 21. Membuat Visualisasi Heatmap.

```
[17]
✓ 0d df_train['STATUS KELULUSAN'].value_counts()
```

	count
STATUS KELULUSAN	
0	206
1	163

dtype: int64

Gambar 22. Menghitung jumlah setiap kategori dalam kolom STATUS KELULUSAN.

```
[18] # Membuat diagram batang untuk status kelulusan
✓ 0 d plt.figure(figsize=(8, 6))
```

Gambar 23. Membuat Figure.

```
sns.countplot(x='STATUS KELULUSAN', data=df_train, palette='Set2')
```

Gambar 24. Membuat Count Plot.

```
# Menambahkan judul dan label
plt.title('Distribusi Status Kelulusan', fontsize=16)
plt.xlabel('Status Kelulusan (1 = Terlambat, 0 = Tepat Waktu)', fontsize=12)
plt.ylabel('Jumlah', fontsize=12)
```

Gambar 25. Menambahkan Judul dan Label.

```
# Menambahkan legend (adjusted labels for correctness)
plt.legend(title='Status Kelulusan', labels=['Tepat Waktu (0)', 'Terlambat (1)'], loc='upper right')
```

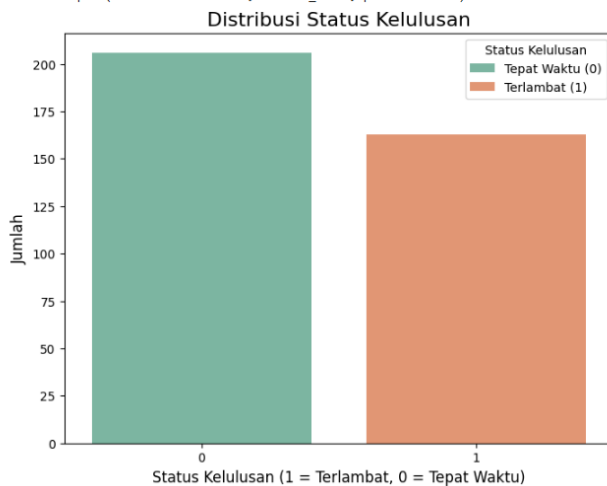
Gambar 26. Menambahkan Legend.

```
# Menampilkan plot
plt.show()
```

... /tmp/ipython-input-3222520158.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='STATUS KELULUSAN', data=df_train, palette='Set2')
```



Gambar 27. Menampilkan Plot.

```
[19] # Menentukan Pembagian fitur dan target, mana yang kan menjadi fitur dan mana yang menjadi target
x_train = df_train.drop(columns=['STATUS KELULUSAN'])
y_train = df_train['STATUS KELULUSAN']
```

Gambar 28. Pemisahan Data Training.

```
[20]
✓ 0 d # data validasinya
x_test = df_test.drop(columns=['STATUS KELULUSAN'])
y_test = df_test['STATUS KELULUSAN']
```

Gambar 29. Pemisahan Data Testing (Validasi).

```
[21]
✓ 0 d x_train.head()
```

	JENIS KELAMIN	STATUS MAHASISWA	UMUR	IPS 1	IPS 2	IPS 3	IPS 4	IPS 5	IPS 6	IPS 7	IPS 8	IPK
0	0	1	28	2.76	2.80	3.20	3.17	2.98	3.00	3.03	0.0	3.07
1	0	0	32	3.00	3.30	3.14	3.14	2.84	3.13	3.25	0.0	3.17
2	0	1	29	3.50	3.30	3.70	3.29	3.53	3.72	3.73	0.0	3.54
3	0	0	27	3.17	3.41	3.61	3.36	3.48	3.63	3.46	0.0	3.41
4	0	1	29	2.90	2.89	3.30	2.85	2.98	3.00	3.08	0.0	3.09

Gambar 30. Melihat X_train.

```
[22]
✓ 0 d y_train.head()
```

	STATUS KELULUSAN
0	1
1	1
2	1
3	1
4	1

dtype: int64

Gambar 31. Melihat y_train.

```
[23]
✓ 0 d # Normalisasi features after splitting
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Gambar 32. Normalisasi Features (StandardScaler).

```
[24]
✓ 0 d      # Melatih Model
           from sklearn.neighbors import KNeighborsClassifier

           model = KNeighborsClassifier(n_neighbors=3)
           model.fit(x_train_scaled, y_train)
```

▼

KNeighborsClassifier

1 2

KNeighborsClassifier(n_neighbors=3)

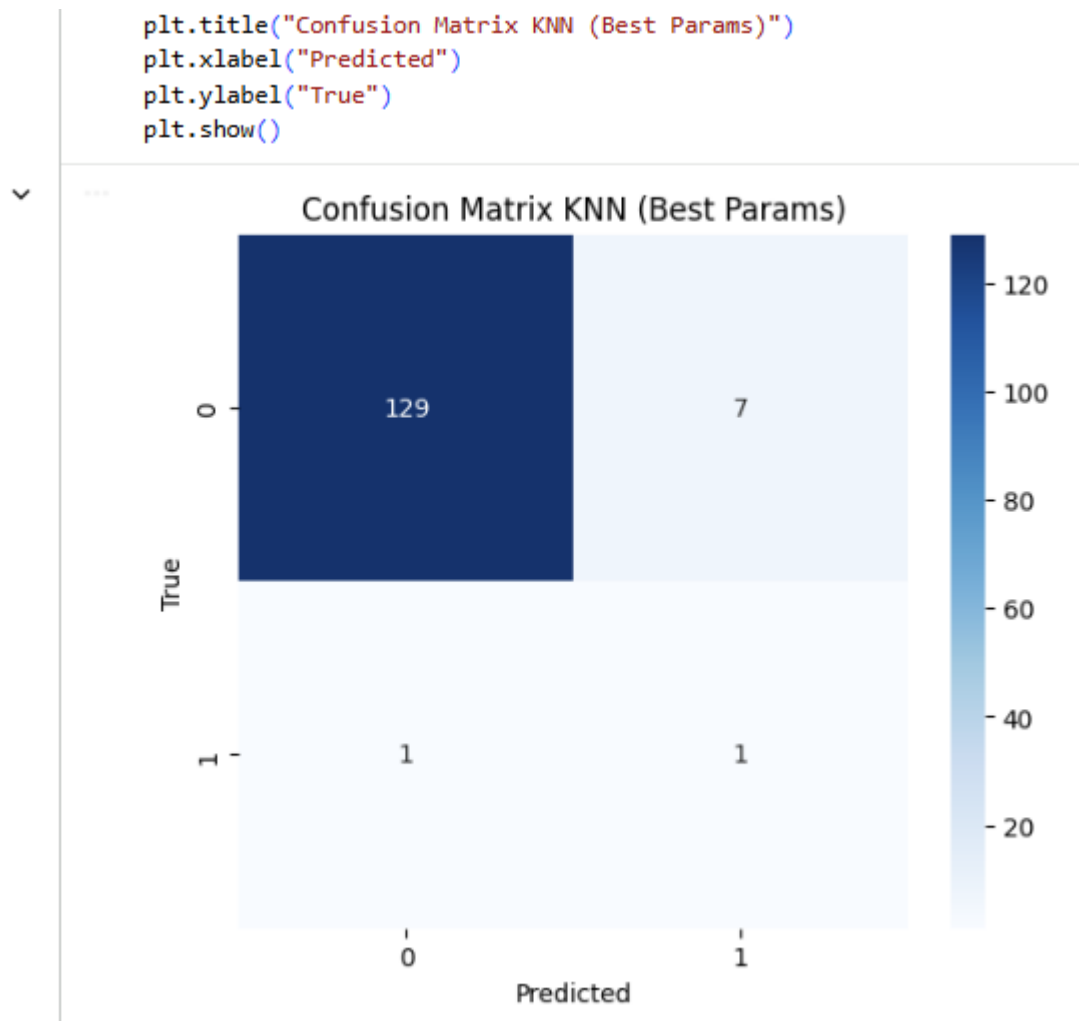
Gambar 33. Melatih Model KNN.

```
[40]
✓ 0 d      cm = confusion matrix(y test, y pred)
```

Gambar 34. Membuat Confusion Matrix.

```
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, cmap='Blues', fmt='d')
```

Gambar 35. Visualisasi dengan Heatmap.



Gambar 36. Label dan Tampilan.

```
[27]
✓ 0d # Untuk menangani ketidakseimbangan kelas
print("Before SMOTE:")
print(y_train.value_counts())
```

Gambar 37. Cek Distribusi Sebelum SMOTE.

```
sm = SMOTE(random_state=42, k_neighbors=1) # Set k_neighbors to 1
x_train_smote, y_train_smote = sm.fit_resample(x_train_scaled, y_train)
```

Gambar 38. Aplikasi SMOTE.

```
print("After SMOTE:")
print(y_train_smote.value_counts())
```

```
*** Before SMOTE:
STATUS KELULUSAN
0    206
1    163
Name: count, dtype: int64
After SMOTE:
STATUS KELULUSAN
1    206
0    206
Name: count, dtype: int64
```

Gambar 39. Cek Distribusi Setelah SMOTE.

```
[28]
✓ 6 d # Mencari parameter terbaik untuk digunakan
param_grid = {
    'n_neighbors': list(range(1, 31)),
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}
```

Gambar 40. Definisi Parameter Grid.

```
grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5, n_jobs=-1)
grid.fit(x_train_smote, y_train_smote)
```

Gambar 41. GridSearchCV.

```
print("Best Params:", grid.best_params_)
print("Best Score (CV):", grid.best_score_)

Best Params: {'metric': 'manhattan', 'n_neighbors': 19, 'weights': 'distance'}
Best Score (CV): 0.9029973552747576
```

Gambar 42. Hasil Terbaik.